Messaging service: architecture guide

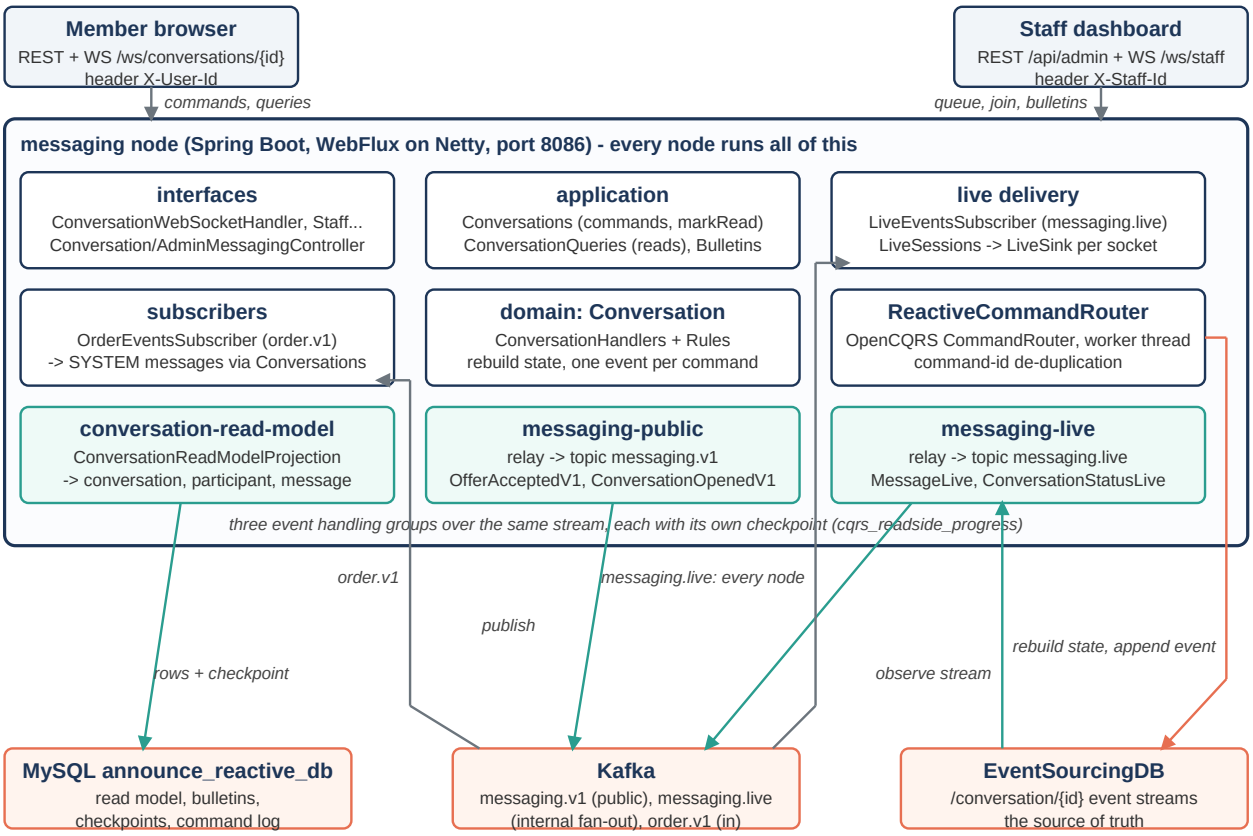
How the components of the announce chat service work together. announce-reactive, module **messaging**, branch feat/messaging, 2026-09-29. Companion to doc/messaging-context.md (functional guide) and doc/messaging-architecture.md (tuning surface).

1. What the service does

The messaging service carries the chat of the marketplace: a buyer talks to the seller of an announce (a **listing** conversation), a member talks to staff (a **support** conversation), and staff receive bulletins on their dashboard. Inside a listing conversation the buyer can make a price **offer**; when the seller accepts it, the order context creates a deal from it. Conversely, every step of a deal (agreed, paid, shipped, settled) appears in the chat as a system message.

The service is built as CQRS with event sourcing on the cqrs-reactive framework. Every change is a **command** that becomes an **event** appended to EventSourcingDB; everything a user reads or receives live is derived from those events afterwards by independent processors. Nothing that a client sees is written directly, with one exception: the read marker of a participant, which is not a domain fact.

2. The service in one picture



Clients reach a node over REST or WebSocket. Commands go down through the application facade, the aggregate and the router into EventSourcingDB. Three processors read the same event stream back and feed MySQL (what clients query), Kafka messaging.v1 (what other services consume) and Kafka messaging.live (what every node pushes to its own sockets). Any number of identical nodes can run; they share nothing but the three stores.

3. The components and what each one owns

3.1 Entry points (package interfaces)

Component	Path	Role
ConversationWebSocketHandler	/ws/conversations/{id}	Identifies the caller from the X-User-Id or X-Staff-Id handshake header, checks that they may attach, then treats every inbound frame as a command (post, offer, accept-offer, decline-offer, read, close, ping) answered with an ack or error frame. Frames everybody sees (message, status) come from the live topic, not from this handler.
StaffWebSocketHandler	/ws/staff?clientType=	The support dashboard channel; needs X-Staff-Id; receives bulletin frames for its client type and sends nothing.
ConversationController	/api/conversations/**	REST fallback for the same commands, plus the inbox (GET /api/conversations), one conversation, the message history (?after=seq) and the read marker.
AdminMessagingController	/api/admin/**	Staff side: the support queue (status=WAITING), join, block, close, bulletins.
Principals		Turns the headers into a Viewer (who asks) or a Sender (who writes); X-Command-Id makes a REST dispatch idempotent.

3.2 Application facades

Component	Role
Conversations	The single door to the command side. Resolves what the aggregate cannot know, i.e. the seller of an announce through the AnnounceLookup port (an HTTP call to announce-management), builds the command and dispatches it through the router. Also raises the read marker directly in the participant row (markRead), which is a read-model write, not an event. Every dispatch is timed (messaging.command.latency).
ConversationQueries	Answers reads from the MySQL read model only, never from the event store: inbox, one conversation, messages after a seq, the support queue, and mayAttach for the socket handshake.
Bulletins	The one thing that is not event-sourced: inserts an announce_bulletin row and publishes a BulletinLive record straight to the live topic through the LiveFeed port.
AnnounceLookup (port)	find(announceId) -> seller and title. Implemented over the framework's discovered, circuit-broken WebClient; tests stub it.

3.3 Domain: the Conversation aggregate

One aggregate per conversation, stored as a stream of events under the subject /conversation/{id}. Its state is deliberately small: type, participants, status, the last message sequence number, the client message ids already seen, the open offer and the accepted offer. Message bodies are not in the state; they live in the read model. Two ids are deterministic on purpose: a listing conversation id is derived from the announce and the buyer, so opening it twice returns the same conversation, and a post carries a client message id, so a retried post returns the original message instead of creating a duplicate.

Command	Rule checked	Event
OpenListingConversation	idempotent when the conversation exists	ConversationOpened (OPEN, buyer + seller)
OpenSupportConversation	member required	ConversationOpened (WAITING, member only)
JoinConversation	staff only; writable; already joined = no-op	ParticipantJoined (WAITING -> OPEN)
PostMessage	duplicate clientMsgId answers the original; writable; sender is a participant; OFFER only in LISTING and only by the buyer; text length	MessagePosted (seq = last + 1)

Command	Rule checked	Event
AcceptOffer / DeclineOffer	seller only; writable; an open offer exists	OfferAccepted / OfferDeclined
CloseConversation	a participant or staff; blocked cannot be closed	ConversationClosed
BlockConversation	staff only	ConversationBlocked

ConversationHandlers holds one handler per command as explicit OpenCQRS definitions. Each handler rebuilds the state from the stream, checks ConversationRules, publishes at most one event and returns the resulting state (or the PostResult with the assigned message id and seq), so the caller can answer immediately without waiting for the read model.

3.4 Framework pieces (cqrs-reactive)

Component	Role
ReactiveCommandRouter	Reactive facade over OpenCQRS's blocking CommandRouter: runs the dispatch on a bounded-elastic scheduler (never on a Netty thread), completes the correlation and causation ids, and de-duplicates commands that carry a command id through the cqrs_command_log table. Handler exceptions become Mono errors; CqrsProblemDetailsHandler maps them for REST.
ReactiveProjectionAdapter / ReactiveProjectionRegistrar	Turn every ReactiveProjection bean into an OpenCQRS event handling processor for its group: the processor fetches the /conversation stream recursively, calls the handler per event and checkpoints after it returns.
R2dbcProgressTracker	Stores each group's checkpoint (last event id) in cqrs_readside_progress in the same MySQL database as the read model.
PublicEventRelay / PublicEventSubscriber	The relay is a projection that maps internal events to versioned public records and sends them with StreamBridge, at least once, keyed by the subject. The subscriber decodes records by their type and version headers.

3.5 The three event handling groups

All three read the same /conversation stream but run as separate processors with separate checkpoints, so they progress independently and each is at-least-once: a crash between the handler and the checkpoint replays the last event, which is why every handler is idempotent.

Group	Class	Output
conversation-read-model	ConversationReadModelProjection	The three MySQL tables: announce_conversation (status, counters, open and accepted offer, last_event_id), announce_conversation_participant (role, last_read_seq) and announce_chat_message (one row per seq).
messaging-public	MessagingRelays.messagingPublicEvents	ConversationOpenedV1 and OfferAcceptedV1 (records of messaging-api) on Kafka topic messaging.v1. This is the contract other services consume; the order context creates an order from OfferAcceptedV1.
messaging-live	MessagingRelays.messagingLiveEvents	MessageLive for every MessagePosted and ConversationStatusLive for joins, offer outcomes, close and block, on Kafka topic messaging.live keyed by the conversation id. Internal to the service.

3.6 Live delivery

Component	Role
LiveEventsSubscriber	Consumes messaging.live with a random consumer group per node and start offset latest, so every running node sees every record, and pushes each one to the sessions this node holds. Measures messaging.live.latency.
LiveSessions	The in-memory registry of sockets open on this node: per conversation id, plus the staff sockets with their client type. Runtime state only; nothing here survives a restart or is shared between nodes. Exposes the session and buffer gauges.

Component	Role
LiveSink	The outbound side of one socket: a serialised, unbounded buffer feeding the WebSocket send. Both the live topic and the session's own ack frames are emitted into it from different threads.
LiveFrames	Encodes the JSON frames a client receives: message, status, bulletin, ack, error, pong.
StreamBridgeLiveFeed	Publishes records that do not come from the event store (bulletins) on the live binding with the same headers the relay uses.

3.7 Inbound from other contexts

OrderEventsSubscriber consumes order.v1 (records of order-api) and turns each deal step into a SYSTEM message in the listing conversation of the buyer and the seller: OrderAgreedV1, OrderPaidV1, OrderShippedV1, OrderSettledV1. It opens the conversation first when it does not exist (a deal from an auction may have had no chat), which is safe because the id derives from announce and buyer. The message's client id is the Kafka event id, so a redelivered record posts nothing twice.

4. How they work together: the life of one message

1 The buyer's browser holds a socket on /ws/conversations/{id} and sends `{"type":"post","clientMsgId":"…","content":"…"}`.

2 ConversationWebSocketHandler parses the frame and calls Conversations.post, which builds a PostMessage command and hands it to ReactiveCommandRouter.

3 The router runs the OpenCQRS handler on a worker thread: it reads the conversation's events from EventSourcingDB, rebuilds the state, checks the rules and writes one MessagePosted event with a precondition that nobody else appended meanwhile. The handler returns a PostResult with the assigned message id and seq.

4 The WebSocket handler emits an ack frame with that result into the session's LiveSink, so the sender sees it at once. This round trip is the ack latency: about 65 ms median in the baseline.

5 Independently, the three processors notice the new event. conversation-read-model inserts the message row and updates the conversation's counters. messaging-live publishes a MessageLive record to Kafka. messaging-public ignores a plain message.

6 Every node's LiveEventsSubscriber receives the record, looks up the sockets of that conversation in its LiveSessions and pushes a message frame to each. Both the buyer and the seller see the message, wherever their sockets are. Clients de-duplicate on the message id, because delivery is at-least-once (the sender receives its own message too).

7 When the seller opens the chat later, GET /api/conversations/{id}/messages?after=seq reads the rows the projection wrote, and a read frame raises the seller's read marker directly in the participant row, without any event.

5. The other flows, same shape

Flow	What happens
Open a listing chat	POST /api/conversations with the announce id: Conversations looks up the seller, sends OpenListingConversation with the deterministic id; ConversationOpened is projected, pushed live as nothing (no socket yet) and relayed as ConversationOpenedV1.
Make and accept an offer	A post with contentType OFFER records an open offer in the state (a newer one supersedes it). The seller's accept-offer produces OfferAccepted: the read model marks the message ACCEPTED and fills accepted_offer_*, the live relay pushes a status frame OFFER_ACCEPTED to both sockets, the public relay publishes OfferAcceptedV1 (conversationId, announceId, buyerId, sellerId, amountCents, currency, acceptedAt) and the order context creates the deal.
A deal progresses	order.v1 records arrive; OrderEventsSubscriber posts a SYSTEM message through the same command path, so it is projected and pushed live exactly like a member's message.

Flow	What happens
Support	OpenSupportConversation starts WAITING with only the member. Staff list the queue with GET /api/admin/conversations?status=WAITING (a read-model query), join with POST .../join, which produces ParticipantJoined and flips the status to OPEN; from there it is an ordinary conversation.
Bulletins	POST /api/admin/bulletins bypasses the event store: a row plus a BulletinLive record on messaging.live; every node pushes it to its staff sockets of the matching client type.
Close and block	A participant or staff closes; only staff block. Both become status frames on the sockets; a blocked conversation cannot be reopened or closed.

6. What each store holds

Store	Content	Who writes	Who reads
EventSourcingDB (docker/cqrs, :3000)	One stream per conversation under /conversation/{id}; the source of truth.	The command handlers, one event per command	The handlers (state rebuild) and the three processors
MySQL announce_reactive_db	announce_conversation, announce_conversation_participant, announce_chat_message (read model); announce_bulletin; cqrs_readside_progress (checkpoints); cqrs_command_log (command ids).	The read-model projection, markRead, Bulletins, the tracker, the router	ConversationQueries, mayAttach
Kafka (:9094)	messaging.v1 (public contract), messaging.live (internal fan-out, keyed by conversation), order.v1 (consumed).	The two relays, StreamBridgeLiveFeed	LiveEventsSubscriber (every node), OrderEventsSubscriber (group messaging), other services

7. Observability

Every node exposes /actuator/prometheus. The messaging.* meters carry a node tag (ANNOUNCE_NODE) and publish p50, p95 and p99 in the process. Package infrastructure.metrics; names in MessagingMeters.

Meter	Where	Meaning
messaging.command.latency (command, outcome)	Conversations.send	dispatch to answer; the ack latency minus the socket hop
messaging.live.latency (record)	LiveEventsSubscriber	event time to the push on this node: ESDB, processor, relay, Kafka, consumer
messaging.live.delivered (record)	LiveEventsSubscriber	frames pushed to local sockets
messaging.projection.delay (group)	MeteredProjection	event written to handled by the group on this node (the lag, per event)
messaging.projection.handler (group, outcome)	MeteredProjection	the handler's own duration
messaging.sessions, .sessions.conversations, .staff_sessions	LiveSessions	sockets on this node
messaging.sink.buffered, .buffered.max, .dropped	LiveSessions / LiveSink	outbound buffer depth and refused frames
r2dbc.pool.*, spring.cloud.stream.binder.kafka.offset	Boot, binder	pool usage and consumer lag, for free

MeteredProjection wraps every projection bean through a BeanPostProcessor, so the framework is untouched. The load driver ChatLoadDriver (test scope) produces the reference load and prints the percentiles plus a scrape:

```
mvn -pl messaging -DskipTests test-compile exec:java \
    -Dexec.classpathScope=test \
    -Dexec.mainClass=org.linlinjava.announce.messaging.load.ChatLoadDriver \
    -Dload.base=http://localhost:8086 -Dload.conversations=20 \
    -Dload.seconds=60 -Dload.intervalMs=3000
```

8. Baseline (2026-09-29) and what it shows

One node, the docker/cqrs stack and the live MySQL on one 8-core host; 20 support conversations, 40 sockets, one post per socket every 3 s for 60 s (800 posts, about 13 per second); all conversations short.

Measure	p50	p95	p99	max
ack latency, driver side (send to ack)	66 ms	131 ms	160 ms	179 ms
fan-out latency, driver side (send to message on either socket)	69 ms	133 ms	163 ms	183 ms
messaging.command.latency PostMessage ok	65 ms	132 ms	166 ms	
messaging.live.latency message	48 ms	103 ms	120 ms	
messaging.projection.delay conversation-read-model	84 ms	122 ms	155 ms	
messaging.projection.handler conversation-read-model	41 ms			

- The command path is where most of the ack latency is, even with short conversations; long ones replay more events per post until the OpenCQRS state cache is on.
- The read-model handler costs about 41 ms per event (two table writes and the checkpoint, each a MySQL round trip), so one partition of that group absorbs about 25 events per second before its lag grows.
- The live path is not the bottleneck at this load: the driver's fan-out latency is only 3 ms above its ack latency.

9. Two findings to fix before tuning

Finding	Where in the picture	Effect	Fix
Concurrent posts in one conversation fail	Step 3 of section 4: two participants post at the same instant; the second append fails EventSourcingDB's precondition (409) and OpenCQRS raises ConcurrencyException; nothing retries.	The WebSocket answers an error frame 500 'internal' (REST answers 409). 12 of 800 posts were lost in the baseline.	Retry with back-off on ConcurrencyException, in ReactiveCommandRouter for every service (or in Conversations.send); until then map it to 409 with a code the client can act on.
Read-your-writes gap on the socket handshake	Step 5 versus the handshake: mayAttach reads the participant rows that the projection has not written yet (p95 about 120 ms, max 770 ms after a join).	A client that connects right after opening or joining is closed with 4403 'not a participant'.	Fall back to the aggregate for the attach check, or make open and join answer only once the read model has caught up; the load driver polls the conversation until it is OPEN.

10. Where to read on

- doc/messaging-context.md: the functional guide (frames, REST, headers, ids).
- doc/messaging-architecture.md: thread and capacity model, every tuning knob, the meters, the experiment order and the baseline.
- doc/cqrs-reactive.md: the framework each package stands on.

- doc/order-context.md section 9: the OfferAcceptedV1 contract with the order context and the recorded deviations.